МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2
по дисциплине: «Объектно-ориентированное программирование»
Тема: «Полиморфизм, система способностей и статус-эффектов в пошаговой игре»

Студент гр. 5382 _____ Рязанов А.Д.

Преподаватель _____ Шестопалов Р.П.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ЦЕЛЬ РАБОТЫ....3
ЗАДАНИЕ....4
1 ОПИСАНИЕ АРХИТЕКТУРЫ....5
1.1 UML-диаграмма классов....5
1.2 Обоснование декомпозиции и борьбы с God-классом....5
1.3 Текстовое описание архитектуры и обоснование архитектурных решений....6
1.4 Используемые паттерны проектирования....6
2 ОПИСАНИЕ КЛАССОВ....8
2.1 Интерфейс IAbility и классы способностей....8
2.2 Интерфейс IStatus и классы статус-эффектов....8
2.3 Обновленный слой управления: GameEngine, LevelLoader, InputHandler. 9
2.4 Класс Visualizer и перечисление Config 9
3 ТЕСТИРОВАНИЕ 11
3.1 Методика тестирования....11
3.2 Результаты тестирования 11
ЗАКЛЮЧЕНИЕ....18

ЦЕЛЬ РАБОТЫ

Целью лабораторной работы является освоение и практическое применение механизмов полиморфизма, наследования и интерфейсов на языке C++. В рамках работы необходимо спроектировать расширяемую систему полиморфных способностей роботов с механикой улучшения при повышении ранга, а также внедрить систему статус-эффектов, взаимодействующую с интерактивными ячейками игрового поля.

ЗАДАНИЕ

В лабораторной работе №2 необходимо реализовать следующие компоненты игры:

1. **Полиморфная система способностей:** Создать интерфейс IAbility с методами Use(), Upgrade() и GetLevel(). Реализовать 4 способности: «Удар вокруг» (увеличивает радиус с уровнем), «Дальний удар» (увеличивает урон), «Лечение» (увеличивает ОЗ) и «Телепортация» (снижает затраты энергии с уровнем).

2. **Механика интерактивной прокачки:** При наборе опыта и повышении ранга робота игрока выводить ncurses-меню для выбора: изучить новую способность или прокачать существующую.

3. **Полиморфная система статусов:** Создать интерфейс IStatus. Реализовать 4 статуса с уникальными правилами суммирования: «Замедление» (запрет движения на ход, стакается по длительности), «Перегрузка» (запрет использования способностей, спадает при попытке каста, несколько работают как одна), «Горение» (урон на старте хода, стакается по длительности), «Щит» (поглощение фиксированного урона, пробитие идет в HP, стакается по прочности).

4. **Интерактивные клетки:** Научить ячейки Cell хранить случайные ловушки-статусы, накладывать один случайный эффект на наступившего робота и разряжаться.

5. **Модификация визуализации:** Добавить поддержку цветов в ncurses и переключение режимов по кнопкам F1-F4 (Стандартный, подсветка врагов, стен и ловушек).

1 ОПИСАНИЕ АРХИТЕКТУРЫ

1.1 UML-диаграмма классов

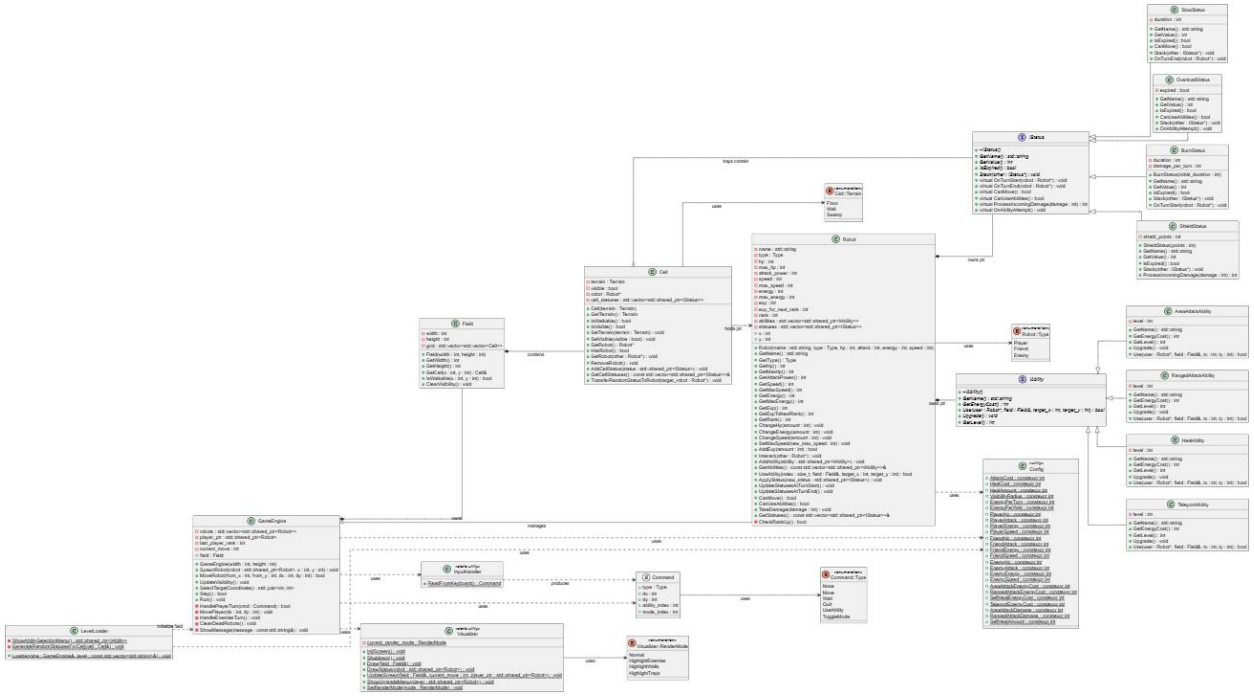


Рисунок 1 — UML-диаграмма классов

1.2 Обоснование декомпозиции и борьбы с God-классом

В первой лабораторной работе класс GameEngine являлся классическим **God Object («Божественным классом»)**. Он одновременно сочетал в себе три несвязанных зоны ответственности:

1. *Низкоуровневый ввод-вывод* (считывание getch() и интерпретация клавиш).
2. *Парсинг данных и инициализация* (чтение строковых массивов карт и конфигурация сущностей).
3. *Игровой цикл*.

В ЛР2 была проведена полная декомпозиция:

- **Command** — пассивный DTO-контейнер, содержащий намерения игрока.
- **InputHandler** — статическая утилита, инкапсулирующая ncurses-ввод.

- **LevelLoader** — фабрика-загрузчик, изолирующая парсинг символов карты и вывод стартового меню способностей.

В результате класс GameEngine стал чистым классом управляющим игровым циклом.

1.3 Текстовое описание архитектуры и обоснование архитектурных решений

Архитектура построена на базе интерфейсов IAbility и IStatus. Слой сущностей (Robot, Cell) не содержит жестко прописанных алгоритмовс способностей. Класс Robot содержит полиморфные композиции std::vector<std::shared_ptr<IAbility>> и std::vector<std::shared_ptr<IStatus>>. Снижение здоровья происходит через TakeDamage(int damage). Этот метод пропускает числовой урон сквозь вектор статусов. Если у робота активен ShieldStatus, урон поглощается его внутренним ресурсом, а остаток передается в ChangeHp.

Ограничения на действия (CanMove(), CanUseAbilities()) вычисляются динамически путем опроса векторов эффектов. Это гарантирует инвариант: поведение робота расширяется внешними классами без модификации кода самого робота.

1.4 Используемые паттерны проектирования

В архитектуре разработанной программы прослеживаются следующие паттерны проектирования:

Полиморфная Композиция: Robot владеет векторами умных указателей std::shared_ptr на базовые интерфейсы.

Фабричный метод: Класс LevelLoader инкапсулирует создание роботов (make_shared) на основе карты. А класс InputHandler создает на основе запроса игрока объект Command.

Разделение ответственности (SOLID): Полностью соблюдены принципы SRP (Единственная ответственность) и OCP

(Открытость/закрытость): добавление новых статусов происходит путем создания новых классов-наследников без изменения ядра движка.

2 ОПИСАНИЕ КЛАССОВ

2.1 Интерфейс IAbility и классы способностей

Чистый абстрактный класс-интерфейс IAbility (файлы IAbility.h, Abilities.h).

Методы:

- GetName() const — возвращает строковое имя.
- GetEnergyCost() const — возвращает цену каста.
- Use(Robot* user, Field& field, int tx, int ty) — выполняет логику способности.
- Upgrade() / GetLevel() — контролируют ранг и силу навыка.

Конкретные классы:

- AreaAttackAbility: наносит урон в радиусе level вокруг пользователя и накладывает на врагов статус OverloadStatus. Применяет безымянные параметры int /*target_x*/ для сохранения сигнатуры без вывода предупреждений компилятора.
- RangedAttackAbility: наносит направленный урон Config::RangedAttackDamage + (level-1)*15 на расстоянии до 3 клеток и накладывает BurnStatus.
- HealAbility: восстанавливает ОЗ. Если здоровье заполнено, избыток (overheal) конвертируется и накладывается на робота в виде ShieldStatus.
- TeleportAbility: перемещает робота в пустую видимую ячейку на расстоянии до 5 клеток, снижая цену энергии на 5 за уровень.

2.2 Интерфейс IStatus и классы статус-эффектов

Интерфейс IStatus (файлы IStatus.h, Statuses.h) задает жизненный цикл и правила наложения эффектов.

Методы:

- OnTurnStart(Robot*) — периодические эффекты в начале хода.
- OnTurnEnd(Robot*) — декремент счетчиков ходов в конце хода.
- ProcessIncomingDamage(int) — фильтрация входящего урона.

- Stack(IStatus* other) — суммирование характеристик эффектов.
- Конкретные классы:
- SlowStatus: блокирует физические перемещения, переопределяя CanMove() const { return false; }. Действует 1 ход. Накапливает длительность ходов при повторном наложении.
- OverloadStatus: переопределяет CanUseAbilities() const { return false; }. Не суммирует длительность. Ловит триггер OnAbilityAttempt(), переводя внутреннее поле expired в true для мгновенной самоликвидации.
- BurnStatus: наносит 5 ОЗ урона через вызов TakeDamage на старте каждого хода. Продлевает длительность ходов при наложении.
- ShieldStatus: хранит прочность shield_points. Вычитает урон из прочности, возвращая 0 во внешний код. При пробитии обнуляется и возвращает остаток урона в здоровье.

2.3 Обновленный слой управления: GameEngine, LevelLoader, InputHandler.

- **InputHandler**: Считывает символы, преобразуя коды стрелочек и функциональных клавиш KEY_F(n) в структуру Command.
- **LevelLoader**: Берет на себя всю логику чтения карт. При парсинге пустых клеток ячейкам с вероятностью 5% выдается одна ловушка, с вероятностью 2% — две (чем больше статусов, тем меньше вероятность). Вызывает стартовое ncurses-меню.
- **GameEngine**: Координирует игровой цикл Run(). Метод MovePlayer очищен от внутренних проверок. Вся проверка замедления перенесена в начало метода HandlePlayerTurn. Если игрок замедлен, нажатие WASD выводит предупреждение и возвращает true, принудительно завершая ход игрока и передавая его ИИ, благодаря чему таймер замедления корректно уменьшается.

2.4 Класс Visualizer и перечисление Config

- Visualizer: В метод InitScreen() добавлены строчки включающие цвета в терминал. Метод Visualizer::Draw() теперь анализирует какой ToggleMode используется и в зависимости от него рисует сцену.
- Config: Расширен константами энергозатрат способностей (AreaAttackEnergyCost, TeleportEnergyCost) и базовой силы эффектов способностей.

3 ТЕСТИРОВАНИЕ

3.1 Методика тестирования

Автоматическое тестирование реализовано на базе фреймворка **Google Test (GTest)**. Проверяются инварианты поглощения урона щитами, механики суммирования ходов замедления, самоликвидации перегрузки и конвертации оверхила способностей.

3.2 Результаты тестирования

```
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Ability
[ RUN      ] Ability.AreaAttackDamageAndOverloads
[       OK ] Ability.AreaAttackDamageAndOverloads (0 ms)
[----------] 1 test from Ability (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running main() from /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Ability.HealConvertsOverhealToShield
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Ability
[ RUN      ] Ability.HealConvertsOverhealToShield
[       OK ] Ability.HealConvertsOverhealToShield (0 ms)
[----------] 1 test from Ability (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running main() from /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Ability.RangedAttackAppliesBurn
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Ability
[ RUN      ] Ability.RangedAttackAppliesBurn
[       OK ] Ability.RangedAttackAppliesBurn (0 ms)
[----------] 1 test from Ability (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running main() from /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Cell.DefaultIsFloorAndWalkable
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Cell
```

```
[ RUN      ] Cell.DefaultIsFloorAndWalkable
[       OK ] Cell.DefaultIsFloorAndWalkable (0 ms)
[----------] 1 test from Cell (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Cell.RobotBlocksWalkable
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Cell
[ RUN      ] Cell.RobotBlocksWalkable
[       OK ] Cell.RobotBlocksWalkable (0 ms)
[----------] 1 test from Cell (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Cell.TransferStatusToRobotOnStep
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Cell
[ RUN      ] Cell.TransferStatusToRobotOnStep
[       OK ] Cell.TransferStatusToRobotOnStep (0 ms)
[----------] 1 test from Cell (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Cell.VisibilityCanBeChanged
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Cell
[ RUN      ] Cell.VisibilityCanBeChanged
[       OK ] Cell.VisibilityCanBeChanged (0 ms)
[----------] 1 test from Cell (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Cell.WallIsNotWalkable
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Cell
[ RUN      ] Cell.WallIsNotWalkable
[       OK ] Cell.WallIsNotWalkable (0 ms)
[----------] 1 test from Cell (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
```

```
[PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Field.CellsStartAsFloor
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Field
[ RUN      ] Field.CellsStartAsFloor
[       OK ] Field.CellsStartAsFloor (0 ms)
[----------] 1 test from Field (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Field.ClearVisibilityResets
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Field
[ RUN      ] Field.ClearVisibilityResets
[       OK ] Field.ClearVisibilityResets (0 ms)
[----------] 1 test from Field (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Field.InvalidSizeThrows
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Field
[ RUN      ] Field.InvalidSizeThrows
[       OK ] Field.InvalidSizeThrows (0 ms)
[----------] 1 test from Field (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Field.ValidSize
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Field
[ RUN      ] Field.ValidSize
[       OK ] Field.ValidSize (0 ms)
[----------] 1 test from Field (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Field.WallBlocksWalkable
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
```

```
[----------] 1 test from Field
[ RUN      ] Field.WallBlocksWalkable
[       OK ] Field.WallBlocksWalkable (0 ms)
[----------] 1 test from Field (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = GameEngine.MoveRobotAttacksEnemy
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from GameEngine
[ RUN      ] GameEngine.MoveRobotAttacksEnemy
[       OK ] GameEngine.MoveRobotAttacksEnemy (0 ms)
[----------] 1 test from GameEngine (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = GameEngine.MoveRobotMovesPlayer
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from GameEngine
[ RUN      ] GameEngine.MoveRobotMovesPlayer
[       OK ] GameEngine.MoveRobotMovesPlayer (0 ms)
[----------] 1 test from GameEngine (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = GameEngine.MoveRobotStopsAtWall
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from GameEngine
[ RUN      ] GameEngine.MoveRobotStopsAtWall
[       OK ] GameEngine.MoveRobotStopsAtWall (0 ms)
[----------] 1 test from GameEngine (0 mstotal)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = GameEngine.SpawnRobotOnEmptyCell
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from GameEngine
[ RUN      ] GameEngine.SpawnRobotOnEmptyCell
[       OK ] GameEngine.SpawnRobotOnEmptyCell (0 ms)
[----------] 1 test from GameEngine (0 ms total)

[----------] Global test environment tear-down
```

```
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = GameEngine.SpawnRobotOnWallDoesNothing
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from GameEngine
[ RUN      ] GameEngine.SpawnRobotOnWallDoesNothing
[       OK ] GameEngine.SpawnRobotOnWallDoesNothing (0 ms)
[----------] 1 test from GameEngine (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Robot.ChangeHpClampsToRange
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Robot
[ RUN      ] Robot.ChangeHpClampsToRange
[       OK ] Robot.ChangeHpClampsToRange (0 ms)
[----------] 1 test from Robot (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Robot.ConstructorSetsValues
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Robot
[ RUN      ] Robot.ConstructorSetsValues
[       OK ] Robot.ConstructorSetsValues (0 ms)
[----------] 1 test from Robot (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Robot.PlayerAttacksEnemy
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Robot
[ RUN      ] Robot.PlayerAttacksEnemy
[       OK ] Robot.PlayerAttacksEnemy (0 ms)
[----------] 1 test from Robot (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Robot.PlayerNotAttacksWithoutEnergy
[==========] Running 1 test from 1 test suite.
```

```
[----------] Global test environment set-up.
[----------] 1 test from Robot
[ RUN      ] Robot.PlayerNotAttacksWithoutEnergy
[       OK ] Robot.PlayerNotAttacksWithoutEnergy (0 ms)
[----------] 1 test from Robot (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Robot.RankUpIncreasesStats
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Robot
[ RUN      ] Robot.RankUpIncreasesStats
[       OK ] Robot.RankUpIncreasesStats (0 ms)
[----------] 1 test from Robot (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Status.BurnDamageAtTurnStart
[==========] Running 1 test from 1 test suite.
[----------] Globaltest environment set-up.
[----------] 1 test from Status
[ RUN      ] Status.BurnDamageAtTurnStart
[       OK ] Status.BurnDamageAtTurnStart (0 ms)
[----------] 1 test from Status (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Status.OverloadBlocksAbilitiesOnce
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Status
[ RUN      ] Status.OverloadBlocksAbilitiesOnce
[       OK ] Status.OverloadBlocksAbilitiesOnce (0 ms)
[----------] 1 test from Status (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Status.ShieldAbsorbsDamageAndExpires
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Status
[ RUN      ] Status.ShieldAbsorbsDamageAndExpires
[       OK ] Status.ShieldAbsorbsDamageAndExpires (0 ms)
[----------] 1 test from Status (0 ms total)
```

```
[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Status.ShieldStackingIncreasesPoints
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Status
[ RUN      ] Status.ShieldStackingIncreasesPoints
[       OK ] Status.ShieldStackingIncreasesPoints (0 ms)
[----------] 1 test from Status (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
Running       main()       from       /usr/src/debug/gtest/googletest-
1.18.0/googletest/src/gtest_main.cc
Note: Google Test filter = Status.SlowBlocksMovementAndStacks
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from Status
[ RUN      ] Status.SlowBlocksMovementAndStacks
[       OK ] Status.SlowBlocksMovementAndStacks (0 ms)
[----------] 1 test from Status (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.
```

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы №2 была успешно решена проблема «Божественного класса» путем декомпозиции GameEngine на слабосвязанные модули InputHandler, LevelLoader и структуру Command. На базе механизмов наследования и чистых интерфейсов C++ была спроектирована полиморфная система способностей роботов и статус-эффектов.

Реализованы уникальные алгоритмы суммирования состояний (по прочности для Щита, по длительности ходов для Замедления/Горения и по триггеру для Перегрузки). Интерактивные клетки поля успешно генерируют и передают эффекты при коллизиях. Модифицированный графический рендерер ncurses поддерживает динамическое переключение цветовых фильтров по кнопкам F1-F4. Все архитектурные инварианты покрыты юнит-тестами Google Test.